Operating Systems and Parallel Computing

Lecture 4

Prof. Matteo Turilli

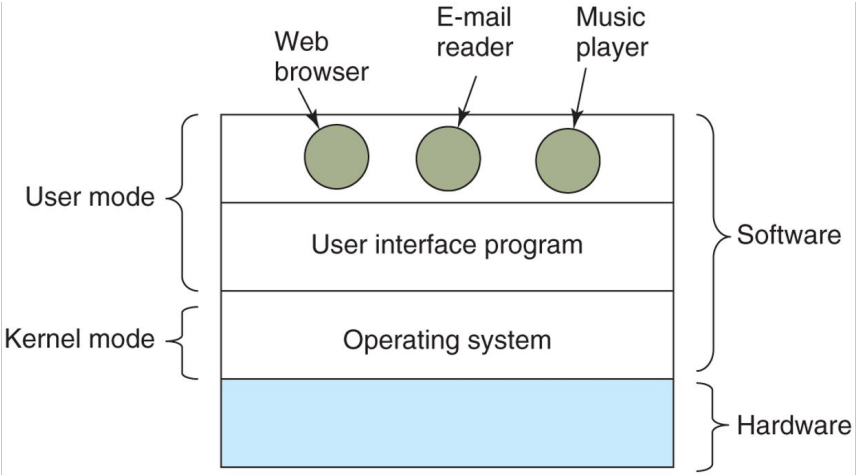
What Is An Operating System (OS)

- Computer: one or more processors, RAM, HD, keyboard, mouse, display, network interfaces,external peripheral, etc.
- No code would ever get written if every programmer had to understand how all these things work in detail.
- Computers executes a type of software called an operating system (OS) that:
  a. Provides a better, simpler, cleaner, model of the computer
  b. Handles managing all the resources just mentioned
- Smartphones and tablets are just computers with a touch screen. They all have OSs.
- Unix-like OSs (*nix) are the most common operating systems in the world: Android, iOS, MacOS, Linux, ChromeOS, Unix, BSD, etc.
- The user interface is not part of the OS. Two types of interfaces: graphical user interface (GUI) and shell.



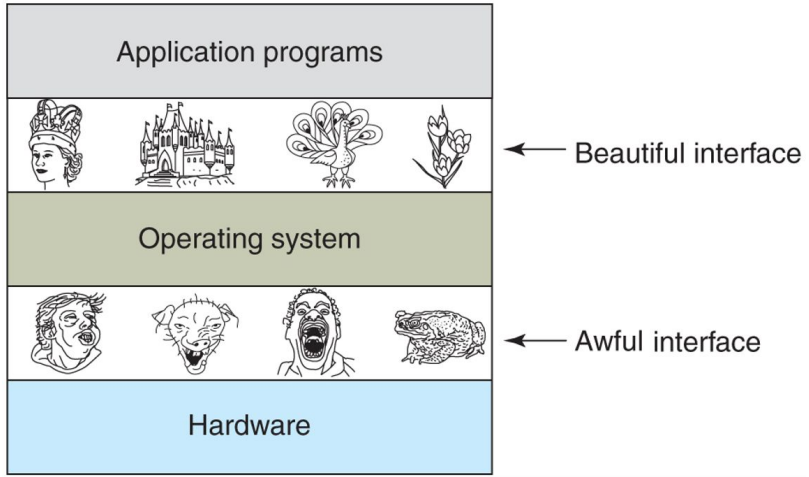
What Is An Operating System (OS)

- Hardware: chips, boards, RAM, drives, disks, keyboard, a monitor, etc.
- Software: two modes of operation modes:
  a. Kernel mode: OS has complete access to all the hardware and can execute any instruction the machine is capable of executing.
  b. User mode: all other software access only a subset of the machine instructions.
- An OS interfaces to the hardware and provides a base for all the other software
- The user interface is the lowest level of user-mode software, and allows the user to start other programs, such as a Web browser, email reader, or music player.
- Applica user mode software relies on the OS.



The Operating System as an Extended Machine

- Hardware programming interfaces are are very complicated, difficult, and inconsistent.
- The OS hides the hardware and present programs (and their programmers) with nice, clean, elegant, consistent, abstractions.
- Example:
  a. SATA hard disk (HD) HW interface spec: 311 pages spec in 2003, 717 in 2011
  b. Disk drivers abstract HW interface exposing functions to read/write disk blocks
  c. OS abstracts away driver interface exposing functions to create/read/write files
- OS implements usable abstractions for application programmers (not end users!)



The Operating System as a Resource Manager

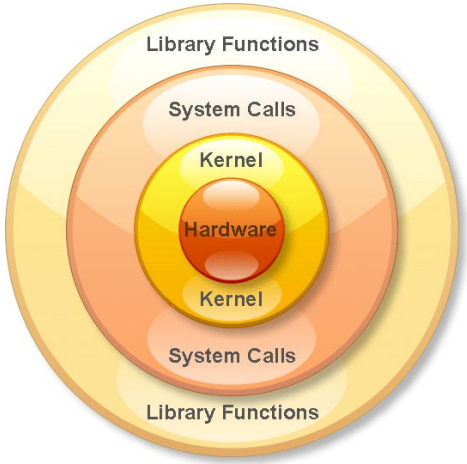
- OS provides for an orderly and controlled allocation of the processors, memories, and I/O devices among the various programs wanting them.
- OS allows for multiple programs to be in memory and run at the same time.
- OS keeps track of which programs are using which resource, to grant resource requests, to account for usage, and to mediate conflicting requests from different programs and users.
- Example:
    a. **Question**: Three programs running on a computer try to print simultaneously on the same printer. What happens?

The Operating System as a Resource Manager

- OS provides for an orderly and controlled allocation of the processors, memories, and I/O devices among the various programs wanting them.
- OS keeps track of which programs are using which resource, to grant resource requests, to account for usage, and to mediate conflicting requests from different programs and users.
- Example:
  a. Three programs running on a computer try to print simultaneously on the same printer. What happens?
  b. Chaos: The first few lines of printout might be from program 1, the next few from program 2, then some from program 3, and so forth.
  c. The OS buffers all the output, and when one program is finished the OS sends its output from the disk file where it has been stored to the printer.
- Resource management includes multiplexing (sharing) resources in two different ways: in time and in space.
  a. Time: different programs or users take turns using the resource (e.g., computing cycles of a CPU)
  b. Space: different programs or users get a part of the available resource (e.g., memory/disk space)

Operating System Interface: System Calls (Syscalls)

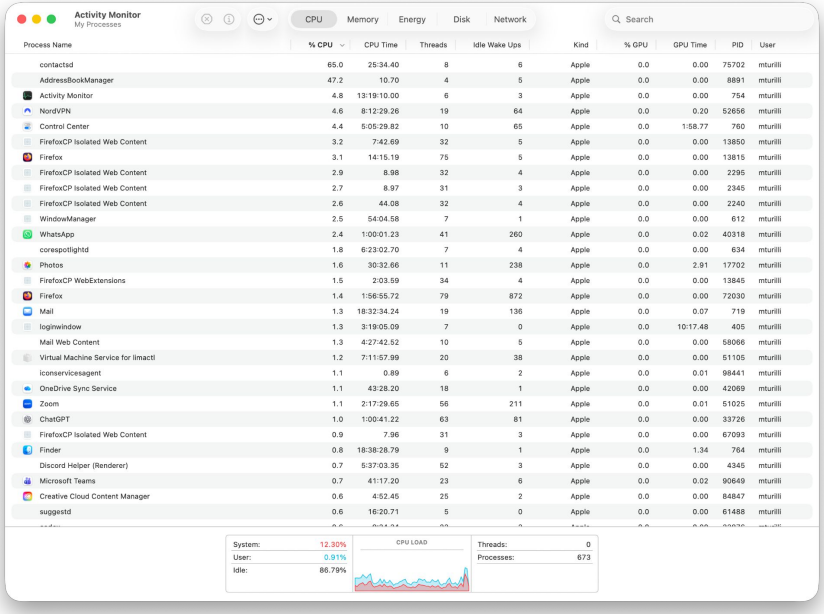
- **Syscall**: a user-space program requests to an OS for a privileged service or hardware interaction.
- How system calls work:
  a. **Load**: A program loads a request in the CPU.
  b. **Mode Switch**: The CPU changes from a safe user mode to a privileged kernel mode.
  c. **Execute**: The kernel looks up the request in a table, checks permissions, and performs the task.
  d. **Return**: The kernel returns the result and switches back to user mode.
- Main types of system calls:
  a. **File Operations**: Open, read, write, and close files
  b. **Process Control**: Create, stop, or wait for programs
  c. **Memory Management**: Request more RAM or map files
  d. **Networking**: Open and manage sockets for data exchange
  e. **Device Control**: Read or write directly to connected hardware devices.

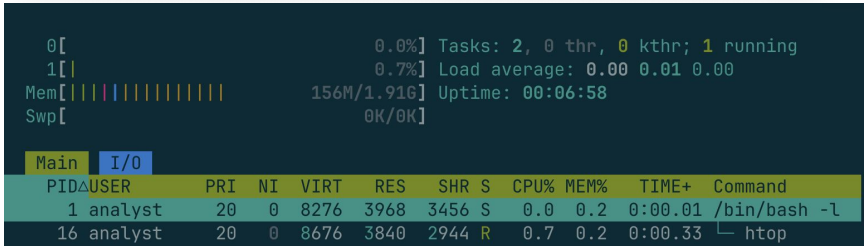


Anatomy of an OS

Programs Vs Processes

- A process is a running program.
- Program != process. The program itself is code saved on disk, a bunch of instructions (and maybe some static data), waiting to “spring into action”.
- The operating system (OS) takes these bytes and gets them running, transforming the program into something useful.
- **Question**: If every program becomes a process when it runs, and it requires a full CPU/core, how do we run multiple programs at the same time?





Time Sharing

- A CPU (or any physical processing unit) can run only one process at a time.
- With only a few physical CPU cores available, how can the OS provide the illusion of a nearly endless supply of cores, i.e., one for each active process?
- OS use **time sharing**: running one process, then stopping it and running another, and so forth.
- Slicing the available processing time across multiple sequential processes and doing it very fast (1000 times/s), gives the illusion of parallel process execution.
- **Question**: Do we have a performance penalty compared to running the same processes, each on a dedicated physical core?
- **Question**: Recall the related notion of space sharing? Do we use it in our laptops and computers?

Mechanisms and Policies

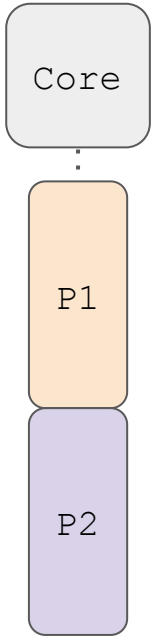
- Implementing time sharing requires both capabilities and rules to make decisions about when to share the available resources across multiple processes.
- **Mechanisms** are low-level methods or protocols that implement a needed piece of functionality, e.g., stop a process and start another one.
- **Policies** are algorithms for making some kind of decision within the OS. For example, given a number of possible programs to run on a core, which program should the OS run?
- Scheduling policies make decisions about processes, e.g., based on:
  a. Historical information, e.g., which program has run more over the last minute?
  b. Workload knowledge, e.g., what types of programs are run
  c. Performance metrics, e.g., is the system optimizing for interactive performance, or throughput?

Concurrency and Parallelism In Multicore Architectures

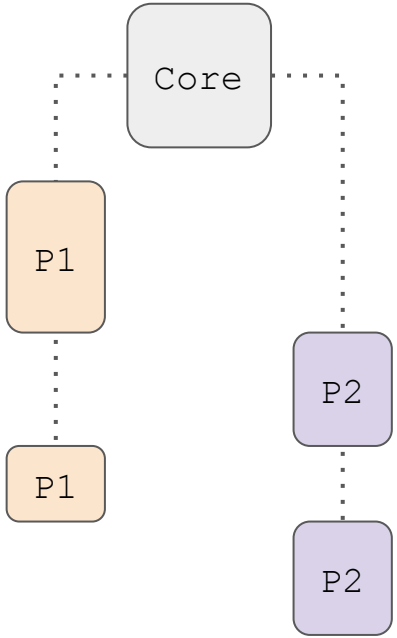
- **Concurrency**: the ability to manage multiple processes whose total execution periods overlap in time, e.g., time-sharing executes P1, then P2, then P1, etc.
- **Parallelism**: running multiple processes at the exact same time on separate hardware cores or processors
- We have heterogeneous processing units (CPU, GPU, TPU, ...) with multiple cores
- OS use time-sharing to achieve concurrency on a single core, and parallelism to do so on multiple cores at the same time.
- This increases the pool of processes that can be active at the same time, but can slow down each executing process due to the performance of each core
- Note: OS using multiple cores != process using multiple cores.
- We will explore in depth the latter in the second part of the course looking at parallel computing and parallel programming

Concurrency and Parallelism In Multicore Architectures

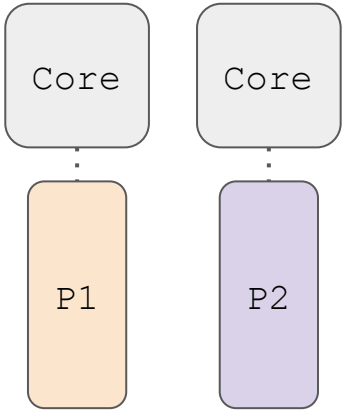
Not concurrent
Not parallel



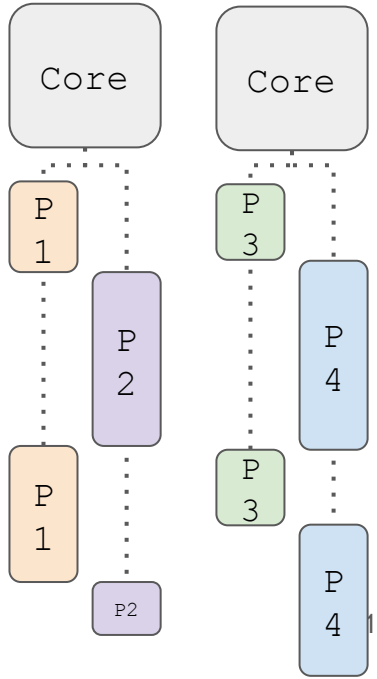
Concurrent
Not parallel



Not concurrent
Parallel



Concurrent
Parallel



Anatomy of a Process

- A process is an activity of some kind with a **program, input, output, and a state**
- When executing, a process has:
  - **Memory**: program's instructions lie in memory, and the data that the running program reads and writes sits in also memory.
  - **Registers**: many instructions explicitly read or update CPU registers
  - **Persistent storage**: I/O information might include a list of the files the process currently has open.
- The OS exposes a process application programming interface (API):
  - **Create**: When we type a command into the shell, or double-click on an application icon, the OS is invoked to create a new process to run that program.
  - **Destroy**: Usually, processes run and exit by themselves when complete; when they don't, the user/OS need a way to kill them
  - **Wait**: wait for a process to stop running, e.g., before collecting output and analyzing it.
  - **Control**: E.g., suspend a process (stop it from running for a while) and then resume it (continue it running).
  - **Status**: get some status information about a process as well, such as how long it has run for, or what state it is in.

The Curious Corner: What Are CPU Registers?

- CPU registers are small, very fast storage locations inside the processor that hold data, instruction information, or addresses so the CPU can work quickly.
- Registers can be categorized based on their functions:
  - General-Purpose Registers (GPR): for arithmetic+logical operations
  - Special-Purpose Registers (SPR): manage specific CPU operations
- Registers are crucial for efficient CPU operation.
- They allow for fast access, data manipulation, and control flow

Reg. Symbol	No. of Bits	Register	Register Function
DR	16	Data Register	Hold operand
AR	12	Address Register	Hold address for Mem
AC	16	Accumulator	Processor register
IR	16	Instruction Register	Hold instruction code
PC	12	Program Counter	Hold address of instruct
TR	16	Temp Register	Hold temp data
INPR	8	Input Register	Holds input char
OUTR	8	Output Register	Holds output char

Process Initialization

- How does a process start? An existing process executes a system call to create the new process, telling the OS which program to run in it.
- In *nix, the system call to create a new process is `fork`:
  - Creates a clone of the calling process, same memory image, environment strings, open files.
  - The child process then executes a similar call to change its memory image and run a new program.
- In Windows, the Win32 system call is `CreateProcess`:
  - Handles both process creation and loading the correct program into the new process
  - Has 10 parameters, including the name of the program to execute.
  - Win32 has about 100 other functions for managing and synchronizing processes and related topics!
- **The parent and child process have distinct memory spaces**. If either process makes a change, the change is not visible to the other process.
- **Note**: in common OSs, the first process to launch when we start our computer is hard-coded into the OS booting image.

Process Termination and Hierarchy

- How does a process stop? usually due to one of the following conditions:
  - **Normal exit (voluntary)**: the process has done its work and executes a system call to tell the OS that it is finished, and its allocated resources can be relinquished.
  - **Error exit (voluntary)**: the process discovers a fatal error, announces the fact, and issues the exit system call as with a normal exit.
  - **Fatal error (involuntary)**: the process **caused** and error, returning an error code or raising a CPU exception, both intercepted by the OS that kills (gracefully) the process.
  - **Killed by another process (involuntary)**: another process with suitable privileges issues a terminating system call for the process.
- Hierarchy of processes:
  - In most OSs, when a process creates another process, the parent process and child process continue to be associated in certain ways
  - UNIX initialization: the `init` process forks off a new process per terminal. If a login is successful, the login process executes a shell to accept commands and each command forks other processes.
  - All the processes in the whole system belong to a single tree, with init at the root.
  - Windows has no concept of a process hierarchy. All processes are equal.

Process States

- A process can be in different states:
  - **Running**: executing instructions on a processor.
  - **Ready**: ready to run but the OS has chosen not to run it at this given moment.
  - **Blocked**: not ready to run until some other event takes place. E.g., when a process initiates an I/O request to a disk, it becomes blocked and thus some other process can use the processor.
- The OS:
  - Moves a process between the ready and running states, i.e., (de)scheduling.
  - Keeps a process blocked (e.g., when it initiates an I/O operation) until some event occurs (e.g., I/O completion)
  - Moves an unblocked process to the ready state again and then to running again, depending on policies.

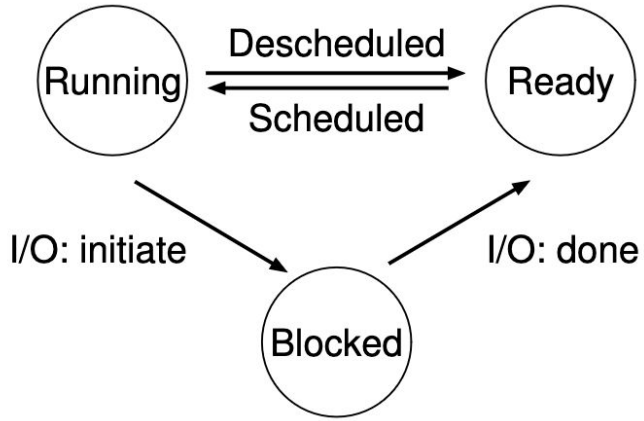


Figure 4.2: **Process: State Transitions**

Examples

Not concurrent, not parallel

Time	$Process_0$	$Process_1$	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	
4	Running	Ready	$Process_0$ now done
5	–	Running	
6	–	Running	
7	–	Running	
8	–	Running	$Process_1$ now done

Figure 4.3: **Tracing Process State: CPU Only**

Concurrent, not parallel

Time	$Process_0$	$Process_1$	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	$Process_0$ initiates I/O
4	Blocked	Running	$Process_0$ is blocked,
5	Blocked	Running	so $Process_1$ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	$Process_1$ now done
9	Running	–	
10	Running	–	$Process_0$ now done

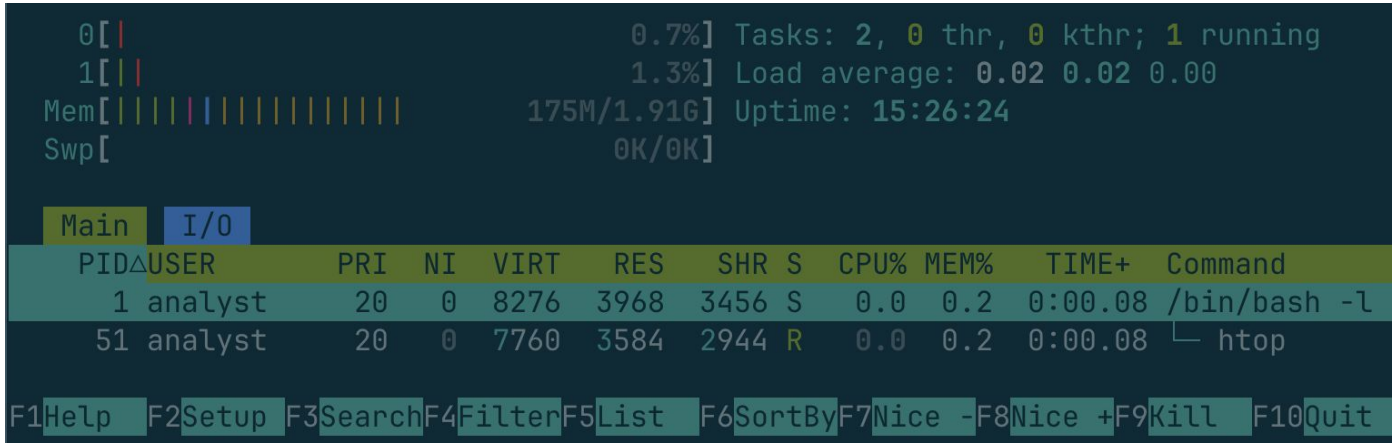
Figure 4.4: **Tracing Process State: CPU and I/O**

This is an interactive class

- Please run our course container and try out all the commands in the next slides
- Do you remember how to run the container? ;-)

Three Commands to See Processes in Your *nix OS

- `ps`: a one-time snapshot of processes. `ps -ef` shows all of them, `man ps` for more information about options/flags
- `top`: a live, updating list, busiest first
- `htop`: like top, but friendlier: colours, scrolling, and easier to read
- Htop running on our container, two processes
- Who owns the two processes?
- Enable the tree view: which process is a parent and which is a child?



Three Commands to See Processes in Your *nix OS

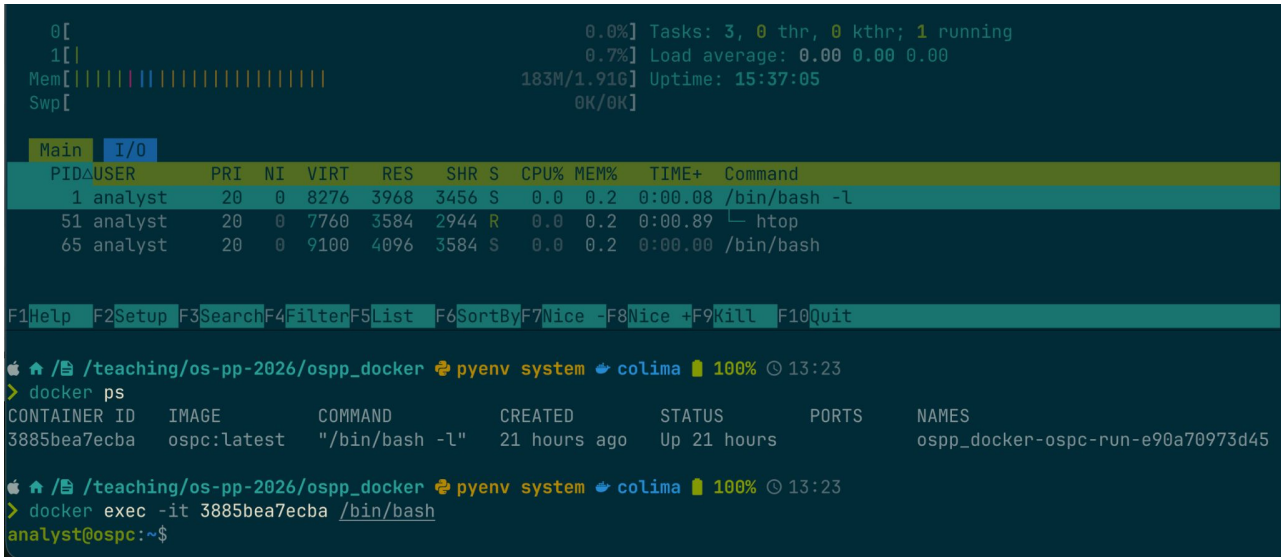
- Open a second shell with:
  - `docker ps`

```
> docker ps
CONTAINER ID   IMAGE          COMMAND          CREATED         STATUS         PORTS     NAMES
3885bea7ecba   ospc:latest    "/bin/bash -l"   21 hours ago    Up 21 hours              ospp_docker-ospc-run-e90a70973d45
```

  - `docker exec -it 3885bea7ecba /bin/bash`
- Interpret the result:

Study `reading_htop.pdf`

Warning: part of midterm quiz



Python and Processes

- To act on processes in a program, Python has psutil.
- psutil.Process() is the current process:
  - It exposes the same properties of (h)top: .pid, .ppid(), .memory_info(), .status(), .parent().
  - We use it to walk the process tree and to watch our own memory grow, live.
- Write this script with `vi ~/work/python_process_info.py`:

```
import psutil
me = psutil.Process()                    # this process
print(me.pid, me.ppid(), me.status())    # id, parent id, state
print(me.memory_info().rss / 1e6, "MB")  # real memory in use
```

- Run it with `python ~/work/python_process_info.py`:

```
analyst@ospc:~$ python ~/work/python_process_info.py
82 65 running
14.176256 MB
```

Exercises and Homework

- Open two terminals in the container, then:
  - In one, run htop. In the other, start a slow job: `sleep 300 &` and find it in htop by name.
  - What is its PID? What is its state (the S column)? Why is it not ‘R’?
  - Use psutil to print your kernel’s PID, parent PID, and memory.
  - Walk up the tree with .parent() until you reach PID 1. Who are your ancestors?
- Before next session:
  - Get comfortable in htop: sort by CPU and by memory, find a process, note its PID and state.
  - Remember: a program is a file; a process is that file running, with a PID, memory, and a state.
  - Next time: the kernel and system calls, what a process actually asks the operating system to do, watched live with strace.